

Back to 2048

Rappels : principes du jeu

À partir d'une grille carrée de côté 4, les joueurs doivent fusionner des puissances de 2 jusqu'à faire apparaître la valeur 2048, en choisissant de déplacer l'ensemble des tuiles dans une direction.

Structure :

- Grille : 4×4 cases
- Tuiles : valeurs qui sont des puissances de 2 (2, 4, 8, 16, 32, 64, 128, 256, 512, ...)

Déroulement d'un tour

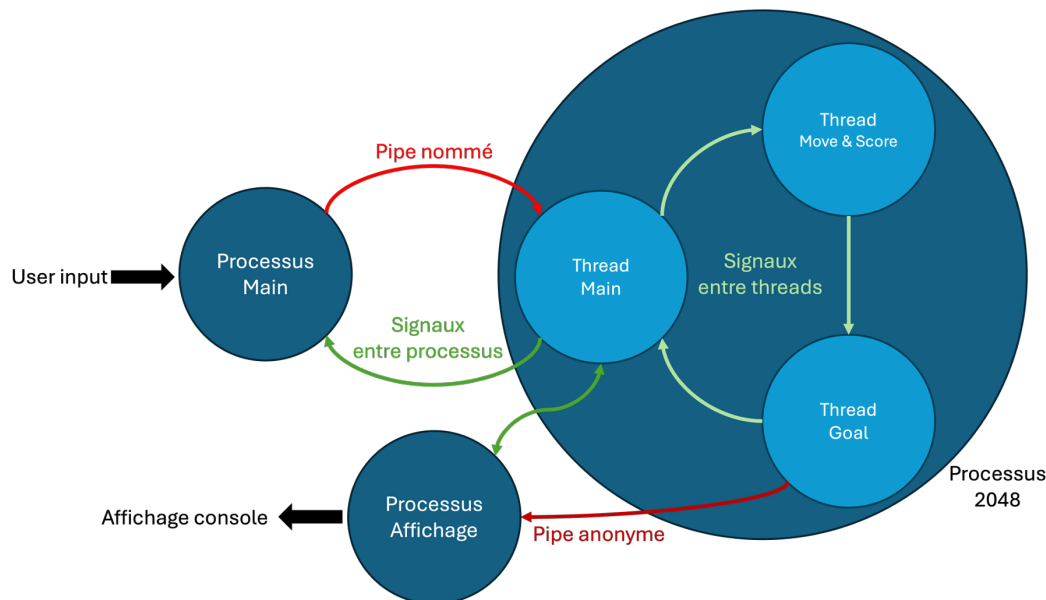
- 1) Apparition d'une tuile : au début et après chaque coup valide, une nouvelle tuile (valeur 2 ou 4) apparaît aléatoirement sur une case vide.
- 2) Déplacement : le joueur choisit une direction (↑, ↓, ←, →). Toutes les tuiles glissent dans cette direction jusqu'à rencontrer un bord ou une autre tuile. Un coup valide est un coup qui entraîne le déplacement d'au moins une tuile.
- 3) Fusion :
 - a) Deux tuiles adjacentes de même valeur fusionnent en une seule tuile de valeur double.
 - b) Chaque tuile ne peut fusionner qu'une seule fois par coup.
 - c) Le score augmente de la valeur de la tuile créée.

Conditions de fin

- Victoire : Une tuile 2048 est créée. La partie s'arrête.
- Défaite : Il n'est plus possible de réaliser un coup valide.

Rappels : ce que vous avez mis en place

En trinôme, vous avez dû mettre en place le jeu 2048 tel que décrit par l'illustration ci-dessous et les consignes suivantes.



- 1) Un premier processus, le processus « main », est à mettre en place. Son rôle est de récupérer les commandes utilisateurs permettant au jeu de se dérouler (déplacement des tuiles ou abandon). Ces commandes sont ensuite écrites dans un pipe nommé vers le second processus, le processus « 2048 », qui réalisera la logique attendue derrière ces commandes.
- 2) Le processus « 2048 » est composé de trois threads ayant chacun un rôle défini et ils devront être synchronisés par des signaux.
 - a) Le thread « principal » doit mettre en place les autres threads, gérer leur synchronisation et gérer les entrées utilisateurs qu'il doit lire sur le pipe nommé partagé avec le processus « main ». Les entrées utilisateurs permettant de déplacer les tuiles seront gérées par le thread « Move & Score » : elles doivent ainsi lui être transmises de la manière qui vous semble la plus judicieuse. L'entrée utilisateur associée à l'abandon est gérée par le thread « principal » qui devra alors proprement stopper l'exécution de tous les processus, à l'aide de signaux.
 - b) Le thread « Move & Score » se chargera de réaliser le déplacement des tuiles selon les règles du jeu et aussi, de calculer le nouveau score suite au déplacement. Il partagera ces informations avec le thread « Goal ».
 - c) Le thread « Goal » se chargera de vérifier si la partie en cours est terminée et de quelle manière (victoire ou défaite). Il a également pour responsabilité de transmettre toutes les informations à afficher au processus « affichage » par le biais d'un pipe anonyme.
- 3) Le processus « affichage » affiche l'état du jeu aux utilisateurs dans la console. Il communiquera et se synchronisera avec le processus « 2048 » par le biais de signaux.

Ce que vous allez devoir faire

En trinôme (vous gardez vos groupes), vous allez devoir modifier votre jeu 2048 tel que décrit par les consignes suivantes. Nous vous proposons trois étapes de développement.

Étape 1

Vous allez devoir modifier votre processus « 2048 » et votre processus « Main » de telle sorte que plusieurs parties de 2048 soient jouables en parallèle : cela signifie qu'il sera possible d'**avoir plusieurs processus « Main »** qui auront la charge de récupérer les entrées utilisateur de chacune des parties auxquels ils seront associés, et qu'ils devront les transmettre à **l'unique processus « 2048 »** qui aura la charge de gérer la logique de toutes les parties. Libre à vous de décider si vous souhaitez avoir un unique processus en charge de l'affichage ou non. Vous devrez synchroniser vos processus « Main » et votre processus « 2048 » avec **un unique pipe nommé et des signaux**.

Étape 2

Les données de toutes les parties devront être stockées dans le tas du processus « 2048 », ce qui sera géré par le thread « Main » (allocation et libération). En revanche, vous devrez utiliser **un segment de mémoire partagé, qui ne pourra contenir que les données associées à une unique partie**, pour réaliser le déplacement des tuiles, calculer le nouveau score (thread « Move & Score ») et vérifier si une partie est terminée (thread « Goal »). Autrement dit, toute cette logique ne devra s'appuyer uniquement que sur des données dans le segment de mémoire partagé tel que défini ci-dessus. Puisque ce segment est une ressource partagée par les threads, vous veillerez à **le protéger et à gérer son accès correctement** (section critique, sémaphore, mutex, ...), tout **en prévenant de potentiels interblocages ou pertes de données**.

Étape 3

Le segment de mémoire que vous utilisez peut maintenant contenir les données associées à N parties, N étant défini au lancement du processus « 2048 ». Vous devez alors modifier votre code pour permettre à ce que les données de ces N parties soient mises à jour par les différents threads correctement.

De nombreuses solutions existent : pensez à bien tout documenter !

Contact : ali.ayadi@unistra.fr ; rorhand@unistra.fr